DIGITALHOUSE CLASE EN VIVO 2

ARCA | AGENCIA DE RECAUDACIÓN Y CONTROL ADUANERO

PL/SQL y optimización de consultas

Clase en vivo 2 · 120 minutos

Programa Integral de Analista de Datos (Data Analyst) — ARCA

Escaneá el QR para registrar tu asistencia

Me presento...

1. Directora General en The eLabs: transformación digital e integración de IA.
2. Host del podcast Hack The Future: estrategia y liderazgo, no solo algoritmos.
3. Docente en la Maestría en Ciencia de Datos (Univ. Austral), ITBA y Di Tella.



Apasionada por aplicar IA para mejorar la productividad y las decisiones.

Lo que dejó la clase 1

1 El grano primero

Qué representa cada fila, antes de agrupar.

2 Los estados definen el número

Completadas, devueltas y canceladas no son lo mismo.

3 Un JOIN puede duplicar

Hay que comprobar el grano después de unir.

4 Validar antes de creer

Una consulta puede correr y responder mal.

De la clase 1 nos llevamos una consulta validada y una forma de defenderla.

La evidencia de la clase 1

1 Pregunta

¿Qué decisión habilita?

2 Definición

Métrica, universo, período y estados.

3 Procedimiento

Qué se hizo, en orden.

4 Resultado

El hallazgo, en una frase.

5 Límite

Qué no se puede afirmar.

Hoy se suman el caso de prueba y el costo observado.

Qué vamos a lograr hoy

PROPÓSITO

Convertir una consulta útil en una pieza reutilizable y probada, y mejorar su rendimiento sin cambiar su significado.

RESULTADO OBSERVABLE

Una función documentada, tres pruebas, la consulta antes y después, un control de equivalencia y la lectura de un plan.

→ Retomamos PL/SQL, procedimientos, funciones y optimización de consultas del módulo 2.

Agenda de la clase

Momento	Actividad	Evidencia
Apertura y el problema	Tres reportes, tres números: ¿cuál es el correcto?	Hipótesis de dónde vive la regla
Base simple y siete pasos	PL/SQL en simple y el proceso de trabajo	Los siete pasos y sus errores
Práctica guiada	La función ticket_promedio y una segunda función	Función con casos borde probados
Revisión colectiva y pausa	Función o procedimiento; un error que compila y miente	Diferencia explicada
Optimización con evidencia	El flujo: medir, plan, decidir, una cosa, comparar	Antes y después con equivalencia
Desafío por equipos	Un pedido distinto por equipo, solo con datos de práctica	Plantilla de seis campos
Revisión cruzada y debriefing	Un equipo audita a otro; qué decidimos y qué quedó sin medir	Fortaleza y brecha
Cierre	Tres frases por escrito	Ticket individual

Tres reportes, tres números

521,66

Marketing · todos los pedidos (7)

538,79

Finanzas · solo entregados (6)

617,08

Operaciones · entregados, últimos 12 meses (5)

Mismo cliente (C1042), tres números. ¿Cuál es el correcto?

Antes de empezar: pensar

¿Cuál de los tres números es el correcto? ¿Dónde vive la regla del ticket promedio, y quién es su dueño?

1 Individual

Dos minutos, por escrito.

2 En parejas

Contrastan sus respuestas.

3 Al grupo

Tres respuestas, vinculadas con las clases online (la clase 1 y el módulo 2).

Registramos los desacuerdos; todavía no los resolvemos.

SQL describe, PL/SQL agrega pasos

SQL

- Describe el resultado que querés
- Una consulta, un resultado
- No guarda lógica ni maneja errores

PL/SQL

- Agrega variables, condiciones y manejo de errores
- Vive en un bloque: DECLARE, BEGIN, EXCEPTION, END
- Se guarda con un nombre y se reutiliza

Una función es, en el fondo, una fórmula con nombre.

Función o procedimiento

Pieza	Qué hace	Cómo se usa	En la clase
Función	Devuelve un valor	Dentro de un SELECT	ticket_promedio
Procedimiento	Hace algo: inserta, actualiza	Se invoca (EXEC o CALL)	—
Regla práctica	Un número para usar en una consulta	→ una función	

Si querés un número para usar en una consulta, es una función.

Parámetros, tipos y qué pasa si no hay filas

Parámetros

Lo que entra, con su tipo y, si conviene, un valor por defecto: el cliente, si cuenta solo entregados.

Variables y tipos

v_ticket NUMBER. Con %TYPE, la variable toma el tipo de una columna.

Si no hay filas

Un promedio de cero pedidos da NULL: no es un error, pero hay que decidir qué hacer y dejarlo escrito.

Siete pasos para desarrollar una función

1. **Describir en una línea**
   Qué hace y qué entra y sale.
2. **Crear el objeto**
   CREATE OR REPLACE y cerrar con /.
3. **Confirmar que compiló**
   Si avisa errores, pedir el detalle.
4. **Consultar el catálogo**
   Que exista y esté válido.
5. **Invocar con valores conocidos**
   Un caso del que ya sabés el resultado.
6. **Validar con un SELECT independiente**
   No con la salida del propio objeto.
7. **Probar un caso borde**
   Sin datos, entrada imposible, valor límite.

Siete pasos, siete evidencias.

Errores frecuentes del proceso

Error	Qué pasa	Cómo evitarlo
Creer que se creó, entonces funciona	Queda inválido y falla al usarlo	Paso 3: confirmar la compilación
No mirar el catálogo	No se sabe si existe ni si es válido	Paso 4: consultarlo
Validar con la propia salida	El error de lógica se confirma solo	Paso 6: un SELECT independiente
Probar solo el caso feliz	Falla con el primer dato raro	Paso 7: al menos un caso borde
Olvidar la barra / al cerrar	En muchas herramientas no se crea	Paso 2: cerrar con /
Copiar la regla en vez de llamarla	Vuelve la regla duplicada	Que cada reporte llame a la función

Cada error tiene un paso que lo atrapa.

Una regla, escrita una sola vez

```
CREATE OR REPLACE FUNCTION ticket_promedio (
    p_id_cliente      IN ventas.id_cliente%TYPE,
    p_solo_entregados IN NUMBER DEFAULT 1
) RETURN NUMBER IS
    v_ticket NUMBER;
BEGIN
    SELECT AVG(unidades * precio_unitario
               * (1 - descuento_pct))
      INTO v_ticket
      FROM ventas
     WHERE id_cliente = p_id_cliente
       AND (p_solo_entregados = 0
            OR estado = 'completada');
    RETURN v_ticket;
END ticket_promedio;
```

1. Una regla, escrita una sola vez
2. Tres reportes, tres llamadas
3. Un solo lugar para corregirla

PL/SQL especificado, no ejecutado en este entorno. Lo que ejecutamos es su equivalente con DuckDB.

Describir y crear la función

```
-- Paso 1 · una línea: calcula el ticket promedio
-- neto de un cliente
CREATE OR REPLACE MACRO ticket_promedio(
    p_id_cliente, p_solo_entregados := 1,
    p_meses := NULL, p_corte := DATE '2025-09-30') AS (
  SELECT AVG(unidades * precio_unitario
             * (1 - descuento_pct))
    FROM ventas
   WHERE id_cliente = p_id_cliente
     AND (p_solo_entregados = 0
          OR estado = 'completada')
     AND (p_meses IS NULL OR (
          fecha > p_corte - to_months(p_meses)
          AND fecha <= p_corte)))
```

1. **Paso 1: describirla en una línea**
2. **Paso 2: crear el objeto**
3. **La regla, escrita una sola vez**

Equivalente ejecutable de la función de Oracle (CREATE OR REPLACE MACRO).

¿Compiló? ¿Existe?

```
-- Paso 3 · ¿compiló? Una macro con un error de tipeo:
CREATE OR REPLACE MACRO macro_con_error(x) AS (
  SELECT AVG(columna_que_no_existe) ...
-- ❌ Binder Error: Referenced column
--    "columna_que_no_existe" not found

-- Paso 4 · ¿existe en el catálogo?
SELECT function_name, function_type
  FROM duckdb_functions()
 WHERE function_name IN ('ticket_promedio',
                         'macro_con_error');
-- solo figura ticket_promedio
```

1. **Si no compila, pedir el detalle del error**
2. **Consultar el catálogo: ¿existe y es válida?**
3. **En Oracle: SHOW ERRORS y USER_OBJECTS**

En Oracle, un objeto con errores se crea y queda inválido; DuckDB directamente no lo crea.

Invocarla con valores conocidos

```
SELECT ticket_promedio('C1042', 0)     AS marketing,
       ticket_promedio('C1042')        AS finanzas,
       ticket_promedio('C1042', 1, 12) AS operaciones;

 marketing | finanzas | operaciones
    521.66 |   538.79 |      617.08

-- los mismos tres números de los reportes:
-- cambió el reporte, no la regla
```

1. Un caso del que ya sabés el resultado
2. Los mismos tres números de los reportes
3. Solo cambian los parámetros

PREDICCIÓN antes de ejecutar: ¿qué tres números esperás?

Validar con un SELECT independiente

```
-- Esta consulta NO usa la función:
SELECT AVG(unidades * precio_unitario
           * (1 - descuento_pct))
  FROM ventas
 WHERE id_cliente = 'C1042'
   AND estado = 'completada';

-- 538.7857  ==  ticket_promedio('C1042')   ✅
```

1. **Validar con una consulta escrita a mano**
2. **No con la salida del propio objeto**
3. **Si no coinciden, hay un error de lógica**

La función no puede validarse a sí misma.

Probar un caso borde

```
caso                          esperado  obtenido
cliente que no existe            NULL      NULL  ✅
corte anterior a todo            NULL      NULL  ✅
todos los pedidos (Marketing)  521.66    521.66  ✅

-- Cada prueba: entrada, esperado y obtenido.
-- En Oracle: un mes sin ventas, un canal
-- inexistente, un mes igual a 13.
```

1. **Al menos un caso borde**
2. **Entrada, esperado y obtenido**
3. **Un caso feliz no alcanza**

El error aparece con el primer dato raro.

Una segunda función: facturacion_mes_canal

Escribí la función que devuelve la facturación neta de un mes y un canal.

- Mes sin ventas → 0
- Mes fuera de 1 a 12 → NULL
- Canal que no existe → NULL

Seguí los siete pasos y corré el verificador cuando creas que está listo.

Revisar sin corregir

Pasan su función

Cada equipo le pasa su código a otro, sin explicarlo.

Quien revisa no corrige

Escribe una prueba capaz de revelar un error.

Se discute

¿Qué caso faltaba? ¿Qué decía el contrato?

Función o procedimiento

Función

- Devuelve un valor
- Se usa dentro de una consulta
- Reutilizable y combinable

Procedimiento

- Ejecuta una acción
- Se invoca; puede recibir y devolver parámetros
- Más control sobre el proceso

Una regla, un solo lugar: cada reporte la llama.

Compila, pero miente

```
CREATE OR REPLACE MACRO ticket_promedio_mal(
    p_id_cliente) AS (
  SELECT SUM(CASE WHEN estado = 'completada'
                  THEN unidades * precio_unitario
                       * (1 - descuento_pct) END)
         / COUNT(*)
    FROM ventas WHERE id_cliente = p_id_cliente)

-- ✅ Se crea sin errores
-- ticket_promedio_mal('C1042')  = 461.82
-- SELECT independiente          = 538.79   ❌
```

1. **Compila: no avisa nada**
2. **Suma lo entregado, divide por todo**
3. **Lo atrapa el SELECT independiente**

PREDICCIÓN: ¿el número será igual, menor o mayor que el de Finanzas?

Antes de salir

Cada equipo guarda su avance y deja escrita una pregunta pendiente.

1

Guardar

Código, consultas y resultados, para retomarlos.

2

Una pregunta pendiente

Por escrito, en el espacio compartido.

3

Las agrupamos

Las más repetidas abren el segundo tramo.

Medir, decidir y comparar



Un plan, de abajo hacia arriba

```
UNGROUPED_AGGREGATE
  Aggregates: sum(#0)
PROJECTION
  unidades * precio_unitario * (1 - descuento_pct)
  ~239,380 rows
SEQ_SCAN
  Table: ventas_grande
  Filters: (strftime(fecha, '%Y-%m') = '2025-03')
  ~239,380 rows
```

1. **Se lee desde la lectura de la tabla hacia arriba**
2. **¿Qué filtro se aplica en la lectura?**
3. **¿Cuál es la operación más costosa?**

Resumen de un plan real de DuckDB (EXPLAIN). Se muestran las operaciones, no los cuadros.

Destrabar el predicado

A · función sobre la fecha

≈ 83 ms

El motor calcula un texto por cada fila.

```
WHERE strftime(fecha, '%Y-%m') = '2025-03'
```

B · rango de fechas

≈ 4 ms

Compara fechas directamente: unas 20 veces más rápida.

```
WHERE fecha >= '2025-03-01' AND fecha < '2025-04-01'
```

Mismo resultado. Un solo cambio.

Según lo que devuelva el filtro

Si devuelve pocas filas

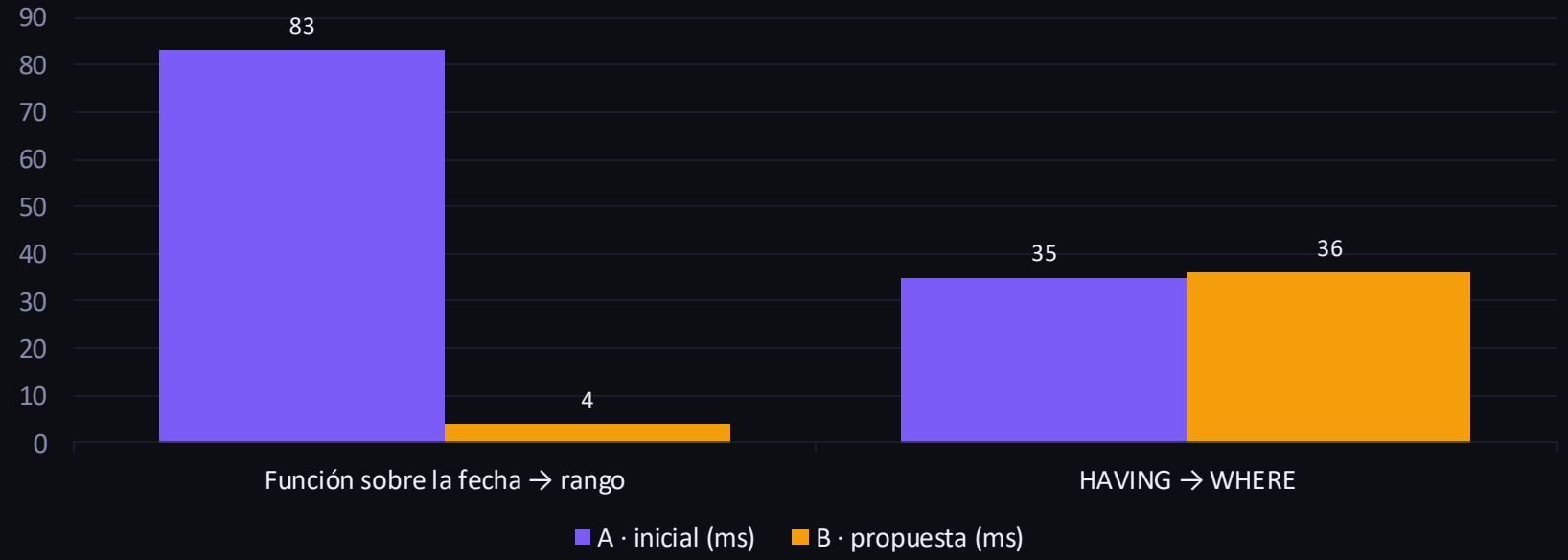
- Escribir el filtro para que el motor lo use: la columna sola, sin funciones
- Mismo tipo de dato en la comparación
- Evaluar un índice, solo con un patrón de acceso que lo justifique

Si no devuelve pocas filas

- Traer solo las columnas que se usan, no SELECT *
- Filtrar antes de agregar: WHERE antes del GROUP BY
- Agregar antes de unir, cuando corresponde

En las dos ramas: una sola cosa por vez, y volver a pedir el plan.

Mismo resultado, otro plan



Un cambio mejoró mucho; el otro no cambió nada. Solo el plan lo mostró.

Buenas prácticas y errores frecuentes

Buenas prácticas

- Medir antes y después con los mismos datos
- Una sola cosa por vez
- Comprobar la equivalencia del resultado
- Declarar el supuesto que sostiene el cambio
- Documentar la versión inicial, la propuesta y el plan

Errores frecuentes

- Optimizar antes de validar
- Cambiar filtros durante la comparación
- Confundir costo estimado con tiempo garantizado
- Crear un índice sin justificar el patrón de acceso
- Cambiar una consulta que el motor ya resuelve bien

El costo del plan es una estimación: se confirma midiendo.

La ficha de evidencia

1

Entrada

Parámetros y datos.

2

Salida

Qué devuelve y de qué tipo.

3

Regla

Cómo se calcula.

4

Caso de prueba

Entrada, esperado y obtenido.

5

Resultado

¿Coincide con el control independiente?

6

Costo observado

Plan y tiempo, sin prometer de más.

Seis campos que se completan al cerrar el desafío.

Un pedido por equipo

CONSIGNA

Mejorá una consulta sin cambiar sus filas ni sus métricas, y demostrá que no cambió su significado.

Trabajan solo con los datos de práctica (datos_desafio_sql_clase2.ipynb) y entregan la plantilla de seis campos.

ROLES ROTATIVOS

Conduce	ordena el trabajo
Ejecuta	escribe la consulta
Valida	controla el resultado
Presenta	cuenta la decisión

Cinco equipos, cinco pedidos

Equipo 1	Categorías	Facturación por categoría: la consulta une primero y agrega después.
Equipo 2	Conteo de ventas	Contar las ventas con COUNT(DISTINCT id_venta) sobre más de un millón de filas.
Equipo 3	Web o tienda	Operaciones de web y de tienda juntadas con UNION.
Equipo 4	Unidades web	Unidades vendidas por web, filtrando con lower(canal).
Equipo 5	Sobre el promedio	Clientes con ticket promedio sobre el general, con una subconsulta por cliente.

La consulta inicial de cada pedido está en la hoja de pedidos.

Cuatro preguntas que te voy a hacer

1

¿Qué resultado debe permanecer igual?

2

¿Dónde está el costo?

3

¿Qué cambio hicieron?

4

¿Cómo probaron que no alteraron el significado?

Si no pueden responder una, esa es su primera pregunta abierta.

Un equipo audita a otro

- El resultado permanece igual: la equivalencia está comprobada.
- El supuesto que sostiene el cambio está verificado, no supuesto.
- El plan antes y después muestra qué cambió.
- El costo observado se explica sin prometer lo que no se midió.
- La plantilla de seis campos está completa.

Devuelven una fortaleza y una brecha, por escrito.

Qué nos llevamos

1 **¿Qué decisión tomamos?**

Y qué alternativa descartamos.

2 **¿Qué error real apareció?**

Uno que compila y miente, o un supuesto sin comprobar.

3 **¿Qué cambió en el plan?**

Qué línea explica la diferencia.

4 **¿Qué quedó sin medir?**

Lo que todavía no podemos prometer.

Escribí tu respuesta en una frase y compartila con tu equipo.

Qué me llevo y qué dejo escrito

CUATRO IDEAS PARA LLEVARSE

1. **Una regla, un solo lugar**
   La función evita la regla copiada en tres reportes.
2. **Siete pasos, siete evidencias**
   Describir, crear, compilar, catálogo, invocar, validar, bordes.
3. **Compilar no es validar**
   Un SELECT independiente atrapa el error que compila.
4. **Optimizar con evidencia**
   Medir, plan, una sola cosa, comparar el resultado.

TICKET DE SALIDA · UNA FRASE POR CADA UNO

Una pieza reutilizable evita…

Completá la frase, por escrito.

Una prueba independiente protege…

Completá la frase, por escrito.

Optimizar con evidencia mejora…

…pero no promete lo que no se midió.

Tres frases por escrito, antes de salir.

Hacia la clase 3

Con datos confiables y piezas reutilizables, el siguiente paso es describirlos e interpretarlos.

Estadística

Python aplicado

Eso introduce la clase 3: estadística y Python aplicado.

DIGITALHOUSE

ARCA | AGENCIA DE RECAUDACIÓN Y CONTROL ADUANERO

PREGUNTAS Y RESPUESTAS

Q&A

¿Qué dudas, casos o ideas quedaron?

¡Muchas gracias!

Laura Sol Franco · Digital House

Programa Integral de Analista de Datos (Data Analyst) — ARCA

Data Analyst



Escaneá para responder la encuesta